Lecture Notes: Dimensionality Reduction

CIC1205 - Machine Learning

Eduardo Bezerra

April 26, 2026

0 Contents

1 Introduction **3**
1.1 Feature Selection vs. Feature Extraction . . . 4
1.2 Why High Dimensionality Can Be Problematic . . . 5

2 Mathematical Background for PCA **6**
2.1 The Data Matrix . . . 6
2.2 Variance and Covariance . . . 7
2.3 The Covariance Matrix . . . 8
2.4 Dot Product, Orthogonality, and Orthogonal Bases . . . 9
2.5 Singular Value Decomposition . . . 9
2.6 Linear Transformations and Eigenvectors . . . 10

3 Principal Component Analysis (PCA) **11**
3.1 Main Intuition . . . 12
3.2 Projection Error . . . 13
3.3 Variance Maximization View . . . 13
3.4 Explained Variance . . . 14

4 The PCA Algorithm **15**
4.1 Step-by-step procedure . . . 15
4.2 Why Centering Matters . . . 16
4.3 Choosing the Number of Components . . . 17
4.4 A Small Worked Example . . . 18
4.5 When PCA Helps . . . 19
4.6 When PCA May Be Less Helpful . . . 19
4.7 Implementation in Scikit-Learn . . . 19

5 Geometric and Probabilistic Intuition for t-SNE and UMAP **20**
5.1 From Distances to Similarities . . . 21
5.2 Why "Local Structure" Matters . . . 22
5.3 A Probability View of Neighborhoods . . . 23
5.4 A Minimal Intuition for KL Divergence . . . 24
5.5 Why a Low-Dimensional Map Is Hard: The Crowding Issue . . . 25
5.6 The Informal Idea of a Manifold . . . 25
5.7 Graphs as a Representation of Local Geometry . . . 25
5.8 What This Background Changes in Practice . . . 26

6 t-SNE: t-Distributed Stochastic Neighbor Embedding **26**
6.1 Motivation: The Limits of Linear Methods . . . 27
6.2 The Core Idea . . . 27
6.3 Why the t-Distribution? . . . 28
6.4 Perplexity: Controlling the Notion of Neighborhood . . . 28
6.5 Properties and Limitations . . . 28

7 UMAP: Uniform Manifold Approximation and Projection **30**
7.1 Motivation and Overview . . . 30
7.2 Core Intuition . . . 30

7.3 Key Hyperparameters 32
7.4 Comparison with PCA and t-SNE 32
7.5 Implementation in Python 33

8 Dimensionality Reduction in a ML Pipeline **34**

9 Summary **35**

Learning Goals

After studying these notes, you should be able to:

- explain why high-dimensional data can be problematic for machine-learning models, especially for distance-based methods such as k-NN;
- distinguish between **feature selection**, **feature extraction**, and **nonlinear embedding**;
- describe PCA as a linear dimensionality-reduction method based on variance maximization and projection-error minimization;
- explain the roles of centering, scaling, covariance, eigenvectors, eigenvalues, and explained variance in PCA;
- choose a reasonable number of principal components using explained variance, while recognizing that explained variance is not the same as predictive relevance;
- explain why t-SNE and UMAP are described as neighborhood-preserving nonlinear embedding methods;
- interpret t-SNE and UMAP plots carefully, avoiding common mistakes involving cluster distances, cluster sizes, densities, and axis meanings;
- explain how dimensionality reduction should be used inside a machine-learning pipeline without causing data leakage.

1 Introduction

In many Machine Learning problems, each example is described by a large number of features. These features may be called *attributes*, *dimensions*, *variables*, *predictors*, or simply *input features*. At first sight, having more features may seem beneficial: the learner receives more information about each example. In practice, however, this intuition is incomplete.

Using all available features may actually hurt predictive performance. Some features may be redundant, some may be noisy, and some may contribute very little useful information relative to the complexity they add. Moreover, several algorithms behave poorly in high-dimensional spaces because the data become sparse and geometric notions such as distance become less informative.

Dimensionality reduction

Dimensionality reduction is the process of representing the original data using fewer variables while attempting to preserve as much relevant information as possible.

There are two broad motivations for dimensionality reduction:

- **Statistical motivation:** reducing noise, redundancy, and overfitting risk.
- **Computational motivation:** reducing memory usage, training cost, and inference cost.

These notes are especially relevant after our discussion of k-NN. Recall that nearest-neighbor methods rely directly on distances between examples. In high dimensions, this

becomes problematic: as dimensionality grows, distances tend to concentrate, and the distinction between "near" and "far" becomes weaker. Dimensionality reduction is one important response to that issue.

In these notes, we study three important dimensionality-reduction methods. We begin with **Principal Component Analysis (PCA)**, a *linear* method that seeks a lower-dimensional representation preserving as much variance as possible. We then discuss **t-SNE** and **UMAP**, two *nonlinear* methods designed mainly to preserve neighborhood structure and reveal geometric organization in low-dimensional visualizations. These methods therefore serve related but distinct purposes: PCA is often used for compression, denoising, and preprocessing, whereas t-SNE and UMAP are especially useful when the goal is to explore possible clusters, manifolds, or local relationships in complex high-dimensional data.

Entry Ticket 1

Why can adding more features sometimes make a model *worse* rather than better? Give at least two distinct reasons.

1.1 Feature Selection vs. Feature Extraction

Not every dimensionality-reduction strategy works in the same way. Broadly speaking, we can distinguish between **feature selection**, **feature extraction**, and, in some cases, **nonlinear embedding**.

- **Feature selection:** keep a subset of the original features and discard the rest. The selected features retain their original meaning and interpretability.
- **Feature extraction:** build new features as combinations of the original ones. These new features no longer correspond directly to the original variables, but they may represent the data more compactly.
- **Nonlinear embedding:** construct a low-dimensional representation whose main goal is to preserve geometric or neighborhood relationships, rather than to retain interpretable feature meanings.

Consider a dataset describing students with 50 variables (grades, attendance, sleep hours, extracurricular activities, and so on). A domain expert might judge that 10 of those variables are the most relevant and discard the rest; that is **feature selection**. By contrast, **Principal Component Analysis (PCA)** would construct 10 new variables, each one being a weighted linear combination of the 50 originals, chosen so that the new variables capture as much variance as possible; that is **feature extraction**.

Later in these notes, we will also study **t-SNE** and **UMAP**. These methods are different in spirit from PCA. Rather than seeking a small set of linear components that summarize variance, they construct a *nonlinear embedding* whose purpose is mainly to preserve neighborhood structure in two or three dimensions. For that reason, they are used primarily for visualization and exploratory analysis rather than for producing interpretable derived features. In practice, however, UMAP is sometimes also used as a nonlinear preprocessing step because it can map new examples into a previously learned embedding.

Feature extraction versus embedding

The distinction between feature extraction and nonlinear embedding is useful, but it is not always absolute in practice.
PCA is a classical *feature extraction* method: it produces explicit linear components, and the same transformation can be naturally applied to new data. The resulting components can also be used as inputs to a downstream predictive model.
t-SNE is usually different. It is mainly a *visualization* method: it constructs a low-dimensional map intended to reveal local neighborhood patterns in the data. The axes of a t-SNE plot should not be interpreted as ordinary derived features, and t-SNE is usually not used as a general-purpose preprocessing step for supervised learning.
UMAP occupies an intermediate practical position. Like t-SNE, it constructs a nonlinear embedding designed to preserve neighborhood structure. However, many implementations of UMAP also provide a `transform` operation, allowing new observations to be mapped into an existing embedding. For this reason, UMAP is sometimes used not only for visualization but also as a preprocessing step in machine-learning pipelines.

1.2 Why High Dimensionality Can Be Problematic

When the number of dimensions grows, the space expands very rapidly. With a fixed number of training examples, the data occupy that space more and more sparsely. This has several consequences:

- local neighborhoods become less meaningful;
- distance-based methods such as k-NN become less reliable;
- the learner may need many more examples to estimate patterns well;
- visualization and interpretation become harder.

To build intuition, consider the following thought experiment. In a 1D interval $[0, 1]$, if we want to cover 10% of the range, we need a segment of length 0.1. In 2D, to cover 10% of the area of the unit square, we need a square of side $0.1^{1/2} \approx 0.32$. In 10D, to cover 10% of the unit hypercube's volume, we need a hypercube with side $0.1^{1/10} \approx 0.79$ — almost the entire range in each dimension. This means that, in 10 dimensions, any "local" neighborhood must already span most of the space to contain a meaningful fraction of the data.

A common phenomenon in many high-dimensional settings is that distances between points tend to *concentrate*: the nearest and farthest distances become less distinguishable. One way to summarize this effect is that the ratio between the largest and smallest pairwise distances may move closer to 1, depending on the data distribution and the distance metric. This makes it harder to distinguish "near" neighbors from "far" ones.

The main message of Figure 1 is that, in many high-dimensional settings, points may start to look almost equally distant. Its purpose is to emphasize the main phenomenon: in high dimensions, many points start to look almost equally distant.

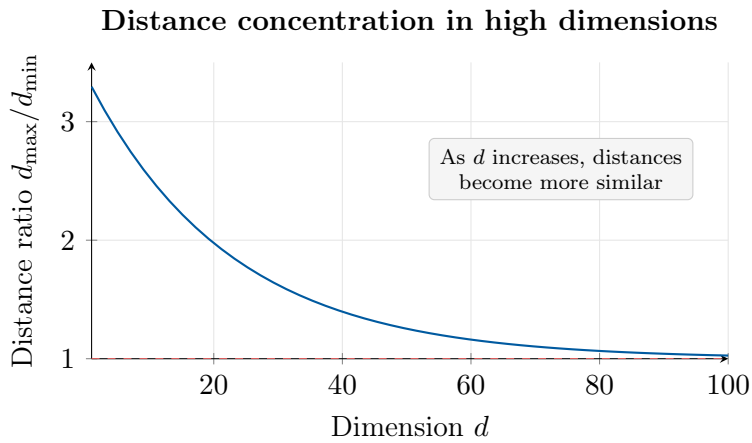


Figure 1: Schematic view of distance concentration. In many high-dimensional settings, the nearest and farthest pairwise distances become less distinguishable. This weakens the meaning of "nearest" neighbors and is one manifestation of the *curse of dimensionality*. The curve is qualitative rather than a universal law; the exact behavior depends on the data distribution and distance metric.

Entry Ticket 2

Why is the curse of dimensionality especially problematic for k-NN? Explain the answer in terms of how k-NN makes predictions.

2 Mathematical Background for PCA

PCA is fundamentally a linear-algebraic method. Before presenting it, we revisit the concepts required to understand its geometry and computation. This background is central for PCA, whereas the later nonlinear methods (t-SNE and UMAP) rely more on neighborhood-based and manifold-based intuitions than on eigendecomposition.

2.1 The Data Matrix

In supervised learning, we are used to a training set of the form

$$\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{m},$$

where m is the number of training examples and each feature vector $\mathbf{x}^{(i)} \in \mathbb{R}^n$ has n features.

In these notes, we focus on the data matrix

$$\mathbf{X} \in \mathbb{R}^{m \times n},$$

that is, the matrix containing only the input attributes (no labels). Each row corresponds to one training example, and each column corresponds to one feature.

A useful statistical viewpoint is to think of each *column* of $\mathbf{X}$ as a random variable and each *row* as an observation of those variables. For example, suppose we have two features: x_1 = number of hours of study and x_2 = final grade obtained. Then different rows of

the data matrix correspond to different students (observed values of those variables), and the two columns are the random variables whose statistical relationship we want to understand.

2.2 Variance and Covariance

The **variance** of a variable measures how spread out its values are around the mean. Given a variable x_j, its sample variance is

$$\mathrm{var}(x_j) = \frac{1}{m-1} \sum_{i=1}^{m} \left(x_j^{(i)} - \bar{x}_j \right)^2 ,$$

where $\bar{x}_j = \frac{1}{m} \sum_{i=1}^{m} x_j^{(i)}$ is the sample mean. A large variance means that observed values are scattered far from the mean; a small variance means they are tightly clustered. The division by $m-1$ (rather than m) yields an unbiased estimator of the true population variance.

The **covariance** between two variables x_j and x_k is

$$\mathrm{cov}(x_j, x_k) = \frac{1}{m-1} \sum_{i=1}^{m} \left(x_j^{(i)} - \bar{x}_j \right) \left(x_k^{(i)} - \bar{x}_k \right) .$$

To understand this formula, think of what each term $(x_j^{(i)} - \bar{x}_j)(x_k^{(i)} - \bar{x}_k)$ is doing. If, for a given example i, both x_j and x_k are above their respective means, the product is positive. If both are below, the product is also positive. If one is above and the other below, the product is negative. Summing over all examples, positive covariance emerges when both features tend to be high (or low) together, and negative covariance emerges when they move in opposite directions.

More precisely:

- **Positive covariance:** the variables tend to increase or decrease together (e.g., hours of study and exam grade).
- **Negative covariance:** when one increases, the other tends to decrease (e.g., hours of exercise and resting heart rate).
- **Covariance near zero:** there is little *linear* relationship between the variables.

Important nuance

Covariance close to zero does *not* mean the variables are independent. It only indicates the absence of a strong *linear* relationship. Two variables could have zero covariance yet be strongly related in a nonlinear way (e.g., $x_2 = x_1^2$ centered at zero). This is a limitation that motivates nonlinear methods such as t-SNE and UMAP, discussed in later sections.

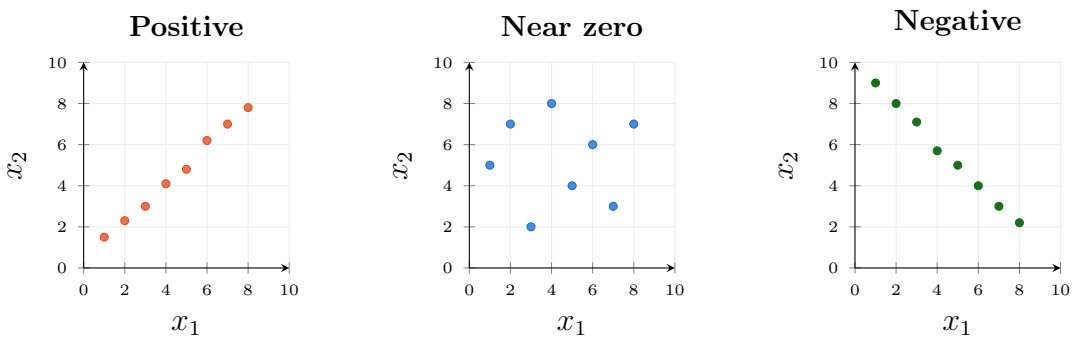


Figure 2: Visual interpretation of covariance. Positive covariance corresponds to an upward trend, negative covariance to a downward trend, and covariance near zero to the absence of a clear linear trend.

2.3 The Covariance Matrix

When the data have multiple variables, we can organize all variances and covariances in a single matrix. For variables $x_1, \ldots, x_n$, the **covariance matrix** is

$$\Sigma = \begin{bmatrix} \mathrm{var}(x_1) & \mathrm{cov}(x_1, x_2) & \cdots & \mathrm{cov}(x_1, x_n) \\ \mathrm{cov}(x_2, x_1) & \mathrm{var}(x_2) & \cdots & \mathrm{cov}(x_2, x_n) \\ \vdots & \vdots & \ddots & \vdots \\ \mathrm{cov}(x_n, x_1) & \mathrm{cov}(x_n, x_2) & \cdots & \mathrm{var}(x_n) \end{bmatrix}.$$

Two important structural facts:

- Σ is **symmetric**: $\Sigma_{jk} = \Sigma_{kj}$ because $\mathrm{cov}(x_j, x_k) = \mathrm{cov}(x_k, x_j)$.
- The **diagonal entries** are variances ($\mathrm{var}(x_j) \geq 0$), so they are always non-negative.
- Σ is **positive semi-definite**: for any vector $\mathbf{v}$, $\mathbf{v}^T \Sigma \mathbf{v} \geq 0$. This ensures that the "amount of variation" along any direction is non-negative, which is geometrically sensible.

Computing Σ from the data matrix. Once the data are mean-centered (each column has mean zero), the covariance matrix can be computed directly as

$$\Sigma = \frac{1}{m-1} \mathbf{X}_{\text{centered}}^T \mathbf{X}_{\text{centered}}.$$

To see why, note that the (j, k) entry of $\mathbf{X}_{\text{centered}}^T \mathbf{X}_{\text{centered}}$ is $\sum_{i=1}^m x_j^{(i)} x_k^{(i)}$. After centering, $x_j^{(i)}$ stands for $x_j^{(i)} - \bar{x}_j$, so this sum is exactly the numerator of $\mathrm{cov}(x_j, x_k)$. Dividing by $m-1$ gives the sample covariance. This matrix-form formula is efficient because it avoids computing each covariance entry individually.

What the covariance matrix summarizes

The covariance matrix tells us, in a single object, both

- how much each feature varies by itself (diagonal entries), and
- how pairs of features vary together (off-diagonal entries).

PCA will use this matrix to identify the most informative directions in the data.

Entry Ticket 3

Why is the covariance matrix always symmetric? What is the interpretation of its diagonal entries? Why can't a diagonal entry be negative?

2.4 Dot Product, Orthogonality, and Orthogonal Bases

Given vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$, their dot product is

$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^{n} u_i v_i = \|\mathbf{u}\| \|\mathbf{v}\| \cos\theta,$$

where θ is the angle between the two vectors. This geometric interpretation is central: the dot product measures how much two vectors "point in the same direction."

Two vectors are **orthogonal** if their dot product is zero, equivalently $\cos\theta = 0$, meaning $\theta = 90°$. In geometric terms, they are perpendicular.

A set of vectors is **orthonormal** if they are mutually orthogonal and each has unit length ($\|\mathbf{v}\| = 1$). Orthonormal directions are especially useful because they separate variation into independent, non-redundant axes.

An **orthogonal matrix $\mathbf{Q}$** is a square matrix whose columns form an orthonormal set. This implies

$$\mathbf{Q}^T\mathbf{Q} = \mathbf{I},$$

which means $\mathbf{Q}^T = \mathbf{Q}^{-1}$. A key property of orthogonal transformations is that they preserve lengths ($\|\mathbf{Q}\mathbf{v}\| = \|\mathbf{v}\|$) and angles between vectors. Geometrically, they correspond to rotations (and reflections) but *never* to stretching. In PCA, orthogonal bases play a central role because the principal components are mutually perpendicular directions in the original feature space.

2.5 Singular Value Decomposition

There are two common ways to compute PCA. The first is conceptually simple: build the covariance matrix and compute its eigenvectors. The second is the one usually used in numerical software: apply **Singular Value Decomposition** (SVD) directly to the centered data matrix. These two views lead to the same principal directions, but SVD is often more stable and efficient in practice.

SVD is one of the most important matrix factorizations in applied mathematics. For any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, SVD guarantees the existence of a factorization

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T,$$

where:

- $\mathbf{U} \in \mathbb{R}^{m \times m}$ is an orthogonal matrix. Its columns $\mathbf{u}_1, \ldots, \mathbf{u}_m$ are called the **left singular vectors**.
- $\mathbf{V} \in \mathbb{R}^{n \times n}$ is an orthogonal matrix. Its columns $\mathbf{v}_1, \ldots, \mathbf{v}_n$ are called the **right singular vectors**.

- $\mathbf{S} \in \mathbb{R}^{m \times n}$ is a rectangular diagonal matrix. Its non-negative diagonal entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ are called the **singular values**.

For our purposes, the most important fact is not the full theory of SVD, but the following connection: when SVD is applied to the centered data matrix, the right singular vectors give the PCA directions.

What do the singular values represent? Intuitively, $\mathbf{A}$ can be seen as a machine that takes vectors in $\mathbb{R}^n$ and maps them to $\mathbb{R}^m$, possibly rotating and stretching them. The singular values tell us how much $\mathbf{A}$ stretches space in each direction: σ_1 is the largest "stretch factor," σ_2 is the next largest, and so on. If some singular values are much larger than others, then $\mathbf{A}$ concentrates most of its action in a few directions, exactly the kind of structure PCA exploits.

Connection to eigenvalues. When we apply SVD to the (centered) data matrix $\mathbf{X}$, the right singular vectors $\mathbf{v}_j$ are precisely the principal component directions, and the squared singular values σ_j^2 are proportional to the eigenvalues of the covariance matrix: $\lambda_j = \sigma_j^2/(m-1)$. This is why, in practice, software libraries compute PCA via SVD rather than explicitly building and diagonalizing Σ: SVD is numerically more stable, especially when m and n are large.

Practical note

In most machine-learning libraries, PCA is implemented numerically using SVD applied directly to the centered data matrix, rather than first computing Σ explicitly. This approach is faster and avoids numerical issues that arise when constructing and diagonalizing the covariance matrix directly. Understanding this connection with SVD is therefore helpful for understanding how PCA is implemented under the hood.

2.6 Linear Transformations and Eigenvectors

To understand PCA geometrically, it is useful to interpret a matrix as a transformation. Suppose, for example, that the covariance matrix is

$$\Sigma = \begin{bmatrix} 9 & 5 \\ 5 & 4 \end{bmatrix}.$$

Multiplying any vector $\mathbf{v}$ by Σ produces a new vector: $\mathbf{u} = \Sigma\mathbf{v}$. Thus, Σ can be seen as a function from $\mathbb{R}^2$ to $\mathbb{R}^2$. In general, such a transformation may both *stretch* and *rotate* vectors.

For example, take the vector $\mathbf{v} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$. After applying Σ:

$$\Sigma\mathbf{v} = \begin{bmatrix} 9 & 5 \\ 5 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 9 \\ 5 \end{bmatrix}.$$

The result $\begin{bmatrix} 9 \\ 5 \end{bmatrix}$ points in a completely different direction from $\mathbf{v}$: the vector was both stretched and rotated. This is the typical behavior for most input directions.

However, there are special directions that do *not* rotate under the transformation; they are only scaled.

Eigenvectors and eigenvalues

A nonzero vector $\mathbf{v}$ is an **eigenvector** of a matrix $\mathbf{A}$ if

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

for some scalar λ. The scalar λ is the corresponding **eigenvalue**.

The equation $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ says that the transformation $\mathbf{A}$ acts on $\mathbf{v}$ by simply scaling it by λ, without changing its direction. A positive λ means the vector is stretched or shrunk; $\lambda < 0$ means it is also flipped; and $\lambda = 0$ means $\mathbf{v}$ is mapped to the zero vector.

Worked example. For the matrix above, the largest eigenvalue is approximately

$$\lambda_1 \approx 12.09.$$

One corresponding unit eigenvector is approximately

$$\mathbf{u}_1 = \begin{bmatrix} 0.85 \\ 0.53 \end{bmatrix}.$$

Indeed,

$$\begin{bmatrix} 9 & 5 \\ 5 & 4 \end{bmatrix} \begin{bmatrix} 0.85 \\ 0.53 \end{bmatrix} \approx \begin{bmatrix} 10.30 \\ 6.37 \end{bmatrix} \approx 12.09 \begin{bmatrix} 0.85 \\ 0.53 \end{bmatrix}.$$

The small discrepancy is due to rounding. The key point: $\mathbf{u}_1$ retains its direction after the transformation; only its length changes (it is scaled by ≈ 12.09).

When $\mathbf{A} = \Sigma$ is a covariance matrix, these eigenvectors identify the *principal directions of variation* in the data. The associated eigenvalues quantify how much variance exists along those directions.

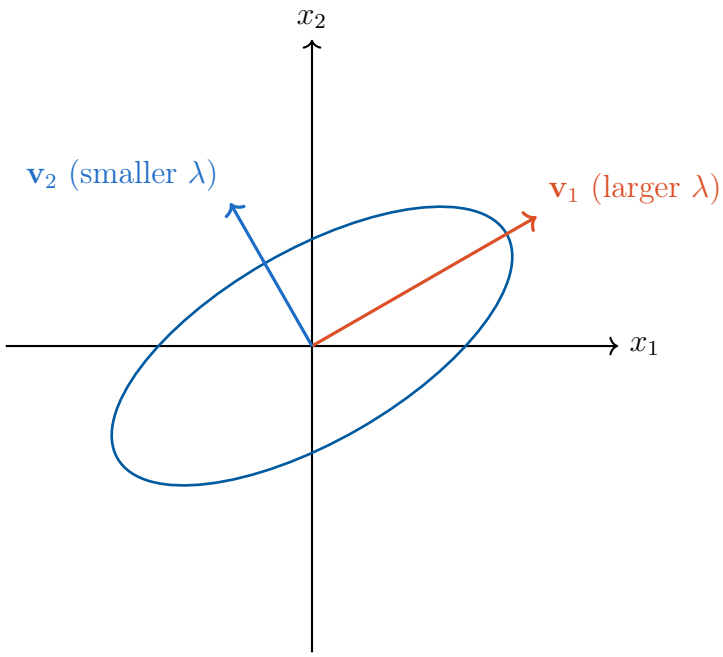
Because covariance matrices are symmetric (and positive semi-definite), their eigenvectors have a particularly nice structure: they can always be chosen to be mutually orthogonal, and all eigenvalues are real and non-negative. This is guaranteed by the *Spectral Theorem* for symmetric matrices. This structural property is one reason PCA is so geometrically clean and computationally tractable.

Entry Ticket 4

If one eigenvalue of the covariance matrix is much larger than the others, what does that suggest about the shape of the data cloud? What would it mean if all eigenvalues were equal?

3 Principal Component Analysis (PCA)

We now turn to the first dimensionality-reduction method in these notes: PCA [Hotelling, 1933]. Unlike t-SNE and UMAP, PCA is a linear method with a precise optimization interpretation in terms of variance maximization and projection-error minimization.



The eigenvectors of a symmetric covariance matrix are orthogonal.
The longer axis corresponds to the direction of greater variance (larger λ).

Figure 3: Geometric interpretation of eigenvectors for a covariance matrix. The data cloud is elongated along one principal direction ($\mathbf{v}_1$) and less spread along the perpendicular direction ($\mathbf{v}_2$). The length of each arrow is proportional to the corresponding eigenvalue.

3.1 Main Intuition

Suppose a dataset has five original variables:

- x_1 = weekly hours of study,
- x_2 = final grade,
- x_3 = weekly hours of video game,
- x_4 = number of rooms at home,
- x_5 = neighborhood crime rate.

If we want to reduce the dimensionality from five to two, PCA does not simply delete three variables. Instead, it constructs two new variables, each one being a *linear combination* of the original five. For instance:

- The first new variable might be a weighted sum of x_1 and x_2 (both related to academic performance), and therefore capture something like "academic engagement."
- The second new variable might combine x_4 and x_5 (both related to living conditions), capturing something like "housing environment."

Crucially, PCA finds these directions *automatically*, without knowing the semantic meaning of the variables. The criterion it uses is purely statistical: among all unit-length linear combinations of the original variables, PCA searches for the direction along which the projected data have maximum variance.

This is the key idea: PCA searches for a new coordinate system in which the most important variation in the data is concentrated in the first few axes.

Principal component

A **principal component** is a direction in feature space along which the projected data have maximum variance, subject to two constraints: (1) the direction must have unit length, and (2) it must be orthogonal to all previously identified components.

Why linear feature extraction is attractive

PCA offers a linear approach to identify directions in the data that retain the greatest amount of variation, without requiring labels or human judgment about which variables matter. Later, we will see that t-SNE and UMAP address a different goal: producing low-dimensional embeddings that better preserve local neighborhood relationships when the data may have nonlinear structure.

3.2 Projection Error

Suppose the original data live in a high-dimensional space and we want to represent them in a lower-dimensional one. Then we must project each point onto a lower-dimensional subspace. Doing so necessarily loses information.

Projection error

The **projection error** is the sum of the squared distances between the original data points and their projections onto the chosen lower-dimensional subspace.

PCA chooses the lower-dimensional subspace so as to **minimize this projection error**. Equivalently, it chooses the subspace that **preserves as much variance as possible**. These two objectives are two sides of the same coin, as we now explain.

Why minimizing projection error and maximizing variance are equivalent. Consider projecting data onto a 1D direction $\mathbf{u}$ (a unit vector). For a centered data point $\mathbf{x}^{(i)}$, the projection onto $\mathbf{u}$ is the scalar $z^{(i)} = \mathbf{u}^T\mathbf{x}^{(i)}$, and the projected point in the original space is $z^{(i)}\mathbf{u}$. By the Pythagorean theorem:

$$\|\mathbf{x}^{(i)}\|^2 = \underbrace{(z^{(i)})^2}_{\text{retained}} + \underbrace{\|\mathbf{x}^{(i)} - z^{(i)}\mathbf{u}\|^2}_{\text{projection error}}.$$

Since $\|\mathbf{x}^{(i)}\|^2$ is fixed, *minimizing the projection error is the same as maximizing the squared projection* $(z^{(i)})^2$, and therefore the same as maximizing the variance of the projected values $z^{(i)}$. This is why both views lead to the same mathematical solution.

3.3 Variance Maximization View

PCA can be described constructively as a greedy process:

- The **first principal component** is the unit vector $\mathbf{u}_1$ that maximizes the variance of the projections $\mathbf{u}_1^T\mathbf{x}^{(i)}$ over all training points.

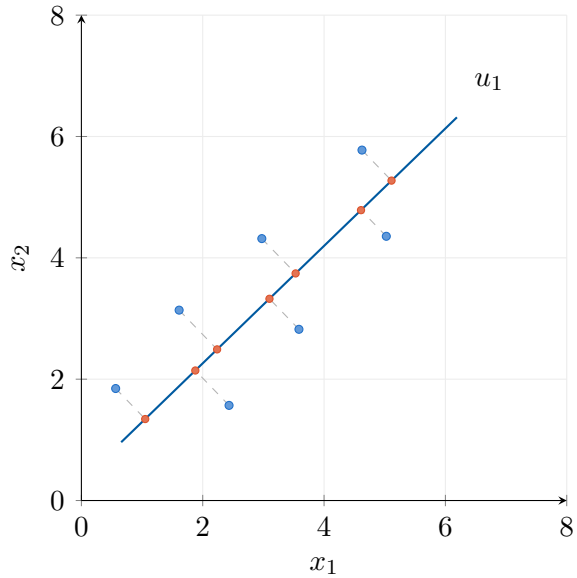


Figure 4: Projection of 2D points onto a 1D subspace. Blue circles are original observations, red circles are their projections onto the line u_1, and dashed segments indicate projection errors. PCA chooses the line that minimizes the sum of squared such errors, which is equivalent to choosing the line along which the projections have maximum variance.

- The **second principal component** is the unit vector $\mathbf{u}_2$, orthogonal to $\mathbf{u}_1$, that maximizes the residual variance not already captured by $\mathbf{u}_1$.
- Each subsequent component is the direction of maximum remaining variance, orthogonal to all previous ones.

It can be shown mathematically that the solution to this optimization problem is given by the eigenvectors of the covariance matrix Σ, ordered by decreasing eigenvalue. The eigenvector associated with the largest eigenvalue λ_1 is $\mathbf{u}_1$, the one with the second largest eigenvalue is $\mathbf{u}_2$, and so on. Moreover, the variance of the data projected onto $\mathbf{u}_j$ is exactly λ_j.

Core idea of PCA

PCA constructs a new orthogonal basis consisting of the eigenvectors of the covariance matrix, ordered by decreasing eigenvalue. Projecting the data onto the first k eigenvectors preserves the maximum possible variance with k linear dimensions, and simultaneously minimizes the total projection error.

3.4 Explained Variance

If the eigenvalues of the covariance matrix are $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$, then the **proportion of variance explained** by component j is

$$\frac{\lambda_j}{\sum_{r=1}^{n} \lambda_r}.$$

This fraction answers the question: "What share of the total variance in the data is captured by the j-th component?"

The **cumulative explained variance** of the first k components is

$$R_k^2 = \frac{\sum_{j=1}^{k} \lambda_j}{\sum_{r=1}^{n} \lambda_r}.$$

This quantity is widely used in practice to choose how many principal components to retain. A common heuristic is to keep enough components so that $R_k^2 \geq 0.90$ or $R_k^2 \geq 0.95$.

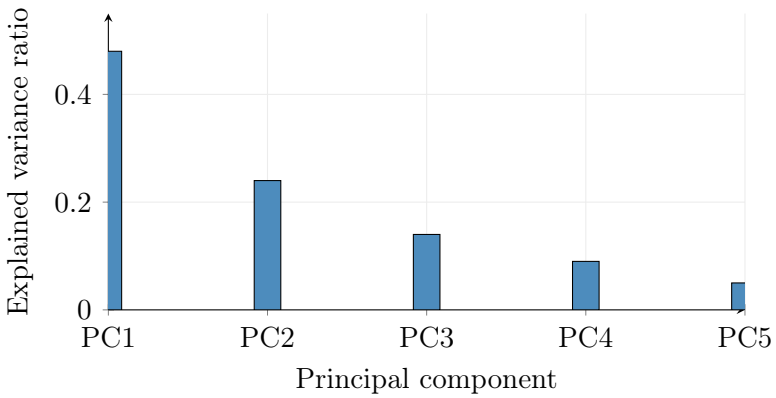


Figure 5: Illustrative explained-variance ratios (a "scree plot"). In this hypothetical example, the first two components explain $0.48 + 0.24 = 0.72$ (72%) of the total variance, and the first three explain $0.48 + 0.24 + 0.14 = 0.86$ (86%). The rate at which explained variance decreases suggests that $k = 3$ or $k = 4$ may be a reasonable choice.

Entry Ticket 5

Suppose the explained-variance ratios of the first three principal components are 0.50, 0.25, and 0.15. What fraction of the total variance is explained by the first two components? And by the first three? If the application requires retaining at least 85% of the variance, how many components should be kept?

4 The PCA Algorithm

Assume that the data matrix is $\mathbf{X} \in \mathbb{R}^{m \times n}$ and that we want a reduced representation with k dimensions, where $k < n$.

4.1 Step-by-step procedure

A standard PCA workflow consists of the following steps:

1. **Center the data.** Compute the mean $\bar{x}_j$ of each feature j and subtract it from every observation:
$$x_j^{(i)} \leftarrow x_j^{(i)} - \bar{x}_j.$$
After this step, every column of $\mathbf{X}_{\text{centered}}$ has mean zero. Centering is *not optional*; its importance is discussed in Section 4.2.

2. **Optionally scale the data.** If features are measured on very different scales (e.g., age in years vs. income in thousands of reais), standardize each feature to unit

variance by dividing by its standard deviation. This prevents features with large numerical scales from dominating. Section 4.2 discusses when this is important.

3. **Compute the covariance matrix.**

$$\Sigma = \frac{1}{m-1} \mathbf{X}_{\text{centered}}^T \mathbf{X}_{\text{centered}} \in \mathbb{R}^{n \times n}.$$

4. **Compute the eigenvectors and eigenvalues of** Σ (or equivalently apply SVD to $\mathbf{X}_{\text{centered}}$). Each eigenvector defines a candidate axis, and the corresponding eigenvalue measures the variance along that axis.

5. **Sort components** by decreasing eigenvalue: $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$.

6. **Keep the first** k **components.** Assemble the k leading eigenvectors into the matrix $\mathbf{W} \in \mathbb{R}^{n \times k}$.

7. **Project the centered data** onto those components:

$$\mathbf{X}_{\text{projected}} = \mathbf{X}_{\text{centered}} \mathbf{W} \in \mathbb{R}^{m \times k}.$$

   Each row of $\mathbf{X}_{\text{projected}}$ is the reduced k-dimensional representation of the corresponding training example.

Projection matrix in PCA

The matrix

$$\mathbf{W} = [\, \mathbf{u}_1 \;\; \mathbf{u}_2 \;\; \cdots \;\; \mathbf{u}_k \,] \in \mathbb{R}^{n \times k}$$

contains the first k principal components as columns. Multiplying the centered data matrix by $\mathbf{W}$ expresses each example in the reduced k-dimensional coordinate system. Note that $\mathbf{W}^T\mathbf{W} = \mathbf{I}_k$ because the columns are orthonormal.

4.2 Why Centering Matters

Centering is essential because PCA measures variance *around the mean*. To see why omitting centering is problematic, consider a simple 1D example.

Suppose the data are $\{10, 11, 10, 12, 11\}$. Their mean is $\bar{x} = 10.8$. If we compute the variance of the centered data, we get the spread of values around 10.8, which is small (the values only fluctuate slightly). Now imagine we had forgotten to center and directly used $\sum_i (x^{(i)})^2 = 556$: this large number is dominated by the mean level (around $10.8^2 \approx 117$ per point), not by the actual fluctuations. PCA would then find a direction that mostly captures the mean location of the data cloud, not its internal variation.

In higher dimensions, the effect is even more pronounced: without centering, the first principal component may simply point from the origin toward the centroid of the data cloud, which is geometrically uninformative.

Connection with feature scaling

Even after centering, features measured in different units may have very different variances. For example, age (range 0–100 years) will have much smaller variance

than income (range 0–1,000,000 reais), simply due to scale. If we apply PCA without standardization, income will dominate the covariance structure and the first principal component will be almost entirely in the direction of income, regardless of whether income is actually the most informative variable. Standardization (dividing each feature by its standard deviation) removes this artifact. However, if all features are already on comparable scales (as with pixel intensities in image data) standardization is typically not needed.

4.3 Choosing the Number of Components

There is no single universal rule for choosing k. Common strategies include:

- **Variance threshold:** keep enough components to explain a target fraction of total variance (e.g., 90% or 95%). The scree plot (Figure 5) displays the cumulative explained variance as a function of k, making it easy to identify an appropriate cutoff.
- **Elbow heuristic:** inspect the individual explained-variance ratios and look for a point where additional components contribute relatively little (an "elbow" in the scree plot).
- **Downstream performance:** treat k as a hyperparameter and select it using cross-validation on the final predictive task.

The last point deserves emphasis. PCA is typically a *preprocessing* step for a downstream model. High explained variance means the compressed representation is faithful to the original data statistically, but it does not guarantee better accuracy on the target task. A direction that captures little variance might still be highly predictive of the label. The best value of k may therefore be determined by validation performance rather than by the variance criterion alone.

Variance is not the same as predictive relevance

In supervised learning, the directions that explain most of the variation in the input data are not necessarily the directions that are most useful for predicting the target. For example, imagine a binary classification problem in two dimensions. Most of the variation in the data occurs horizontally: points are widely spread from left to right. However, the two classes differ mainly by a small vertical shift. In this case, the first principal component will likely be horizontal, because that is where the largest variance is. But this direction may be almost useless for classification. The low-variance vertical direction, although statistically smaller, may be exactly the direction that separates the two classes.
This example illustrates why choosing the number of components only by explained variance can be misleading in supervised tasks. When PCA is used before a predictive model, the number of components should often be treated as a hyperparameter and selected using validation or cross-validation.

Entry Ticket 6

Is maximizing explained variance always the same as maximizing predictive performance in a downstream supervised-learning task? Why or why not? Describe a concrete scenario in which a low-variance direction could be more important for prediction than a high-variance direction.

4.4 A Small Worked Example

Consider a centered dataset in two dimensions whose covariance matrix is

$$\Sigma = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix}.$$

Finding the eigenvalues. We solve $\det(\Sigma - \lambda \mathbf{I}) = 0$:

$$\det \begin{bmatrix} 4-\lambda & 2 \\ 2 & 3-\lambda \end{bmatrix} = (4-\lambda)(3-\lambda) - 4 = \lambda^2 - 7\lambda + 8 = 0.$$

Using the quadratic formula:

$$\lambda = \frac{7 \pm \sqrt{49-32}}{2} = \frac{7 \pm \sqrt{17}}{2}.$$

Therefore $\lambda_1 = \frac{7+\sqrt{17}}{2} \approx 5.56$ and $\lambda_2 = \frac{7-\sqrt{17}}{2} \approx 1.44$.

Finding the eigenvectors. For $\lambda_1 \approx 5.56$, we solve $(\Sigma - \lambda_1 \mathbf{I})\mathbf{v} = \mathbf{0}$:

$$\begin{bmatrix} -1.56 & 2 \\ 2 & -2.56 \end{bmatrix} \mathbf{v} = \mathbf{0} \implies v_2 = 0.78\, v_1.$$

Normalizing: $\mathbf{u}_1 \approx \begin{bmatrix} 0.79 \\ 0.61 \end{bmatrix}$. Similarly, $\mathbf{u}_2 \approx \begin{bmatrix} -0.61 \\ 0.79 \end{bmatrix}$. One can verify $\mathbf{u}_1 \cdot \mathbf{u}_2 = (0.79)(-0.61) + (0.61)(0.79) = 0$: orthogonal.

Interpreting the results.

- The direction $\mathbf{u}_1 \approx \begin{bmatrix} 0.79 \\ 0.61 \end{bmatrix}$ is the first principal component. It points diagonally, indicating that the most variation is shared between both original features (they vary together).
- Most of the data variance lies along $\mathbf{u}_1$.
- If we reduce from 2D to 1D, the best single direction to keep is $\mathbf{u}_1$.

The explained-variance ratios are:

$$\frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{5.56}{7.00} \approx 0.794, \qquad \frac{\lambda_2}{\lambda_1 + \lambda_2} = \frac{1.44}{7.00} \approx 0.206.$$

Thus, one component already explains about 79.4% of the total variance. If 80% is a sufficient threshold for the application, reducing to 1D is justified.

4.5 When PCA Helps

PCA can be especially useful when:

- the dataset has many correlated numeric features;
- a lower-dimensional representation is desired for visualization (typically $k = 2$ or $k = 3$);
- a downstream learner struggles with noise or high dimensionality (e.g., k-NN in very high dimensions);
- storage or computational resources are limited.

4.6 When PCA May Be Less Helpful

PCA may be less useful when:

- interpretability of original features is essential: the principal components are linear combinations of all original variables, making them hard to explain to non-technical stakeholders.
- the target depends primarily on low-variance directions that PCA would discard.
- the structure of the data is *nonlinear*: PCA is a linear method and will not capture curved or manifold-like structures (see t-SNE and UMAP in later sections).
- features are categorical or mixed. PCA assumes numeric variables for which linear combinations are meaningful. Applying PCA directly to arbitrary encoded categories may produce components that are difficult to interpret and sometimes statistically inappropriate.

In such situations, PCA is often a strong first baseline because it is simple, fast, deterministic, and easy to integrate into downstream pipelines.

Entry Ticket 7

Why might PCA hurt predictive performance in some supervised-learning problems, even if it preserves most of the total variance? Describe a concrete scenario in which the information discarded by PCA is exactly the information needed to separate two classes.

4.7 Implementation in Scikit-Learn

In Scikit-Learn, PCA is available through `sklearn.decomposition.PCA`. A typical workflow is:

1. standardize the numeric features if appropriate (using `StandardScaler`);
2. fit PCA on the training data (`pca.fit(X_train)`);
3. transform training and test data (`pca.transform(X)`);
4. fit the downstream model on the transformed training set.

Common constructor arguments include:

- `n_components`: number of components to keep; if a float in $(0, 1)$, it is interpreted as the desired fraction of explained variance (e.g., `n_components=0.95` keeps as many components as needed to explain 95% of the variance).
- `svd_solver`: computational strategy (`'full'`, `'randomized'`, `'arpack'`); the option `'randomized'` is much faster for large datasets when only a small number of components is needed.
- `whiten`: if `True`, rescales each component to unit variance; useful for some downstream classifiers.

After fitting, `pca.explained_variance_ratio_` contains the explained variance fractions in decreasing order, and `pca.components_` contains the rows of $\mathbf{W}^T$ (i.e., the principal component directions as row vectors).

A common and correct pattern is to combine scaling, PCA, and a classifier inside a single `Pipeline` object. This ensures that all preprocessing parameters are fitted within each cross-validation fold, preventing leakage automatically.

The validation or test data are never used to decide the scaling parameters, the principal components, the UMAP embedding, or the classifier parameters.

```python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=0.95)),
    ("clf", LogisticRegression(max_iter=1000))
])

scores = cross_val_score(pipe, X, y, cv=5)
```

Listing 1: Using PCA inside a scikit-learn pipeline to avoid data leakage

In the example shown in Listing 1, each cross-validation fold fits its own scaler and its own PCA transformation using only the training portion of that fold. The validation portion of the fold is transformed using those fitted objects. This prevents data leakage.

5 Geometric and Probabilistic Intuition for t-SNE and UMAP

Unlike PCA, t-SNE and UMAP are not built around covariance matrices, eigenvectors, or orthogonal projections. Their starting point is different. Instead of asking "Along which linear directions does the data vary the most?", they ask a more local question: *which points are neighbors of which other points, and how can that neighborhood structure be represented in a low-dimensional space?*

This difference in perspective is important. PCA is fundamentally a method of *linear feature extraction*, whereas t-SNE and UMAP are methods of *nonlinear embedding*. For that reason, the mathematical ideas that support them are also different. The most important background concepts here are:

- distances and similarities;
- probability distributions over neighbors;
- the distinction between local and global structure;
- the informal idea of a manifold.

The goal of this section is not to provide a full mathematical treatment of information theory, topology, or differential geometry. Instead, it introduces the minimum conceptual background needed to understand what t-SNE and UMAP are trying to preserve, what their plots mean, and what their plots *do not* mean.

5.1 From Distances to Similarities

In many machine-learning settings, we begin with pairwise distances between examples. For instance, if two handwritten-digit images are numerically similar as vectors of pixel intensities, their Euclidean distance may be small. If they are very different, their distance may be large.

However, methods such as t-SNE and UMAP do not work only with raw distances. Instead, they transform distances into **similarities**, that is, numerical measures of how strongly two points should be viewed as neighbors.

Similarity

A **similarity** is a quantity that becomes larger when two points are judged more alike or more strongly connected, and smaller when they are judged less alike or less connected.

This transformation is useful because raw distances are often hard to compare across different regions of the dataset. In one region, a distance of 0.5 may indicate that two points are very close; in another, the same value may not be especially meaningful. By converting distances into neighbor-strength values, the algorithm can focus more directly on *who is near whom*.

A common pattern is:

- very small distance $\Rightarrow$ high similarity;
- moderate distance $\Rightarrow$ intermediate similarity;
- large distance $\Rightarrow$ low similarity.

This is the first conceptual step behind both t-SNE and UMAP. They do not try to preserve all pairwise distances exactly. Instead, they try to preserve the *neighbor structure* induced by those distances.

Key idea

For t-SNE and UMAP, the central object is not a covariance matrix or a projection direction. It is a representation of which points count as neighbors, and how strongly they are connected.

Figure 6 shows the conceptual transition from raw distances to soft neighborhood strengths. Instead of making a rigid yes/no decision about whether another point is a neighbor, nonlinear embedding methods assign stronger weights to closer points and weaker weights to more distant ones. This makes it possible to describe neighborhood structure in a graded way, which is especially useful when the data do not have a simple linear geometry.

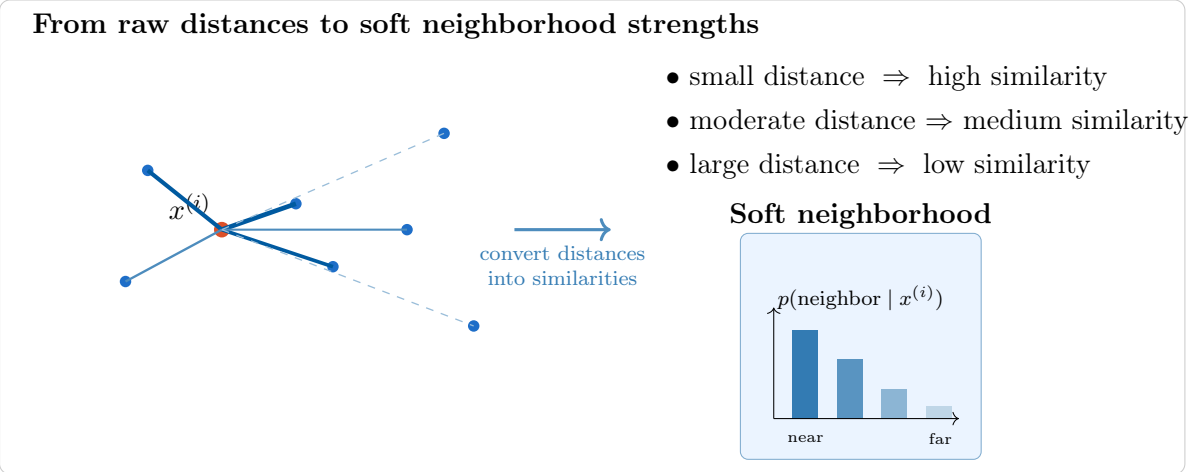


Figure 6: Distances can be converted into soft neighborhood strengths. Relative to a reference point $x^{(i)}$, nearby points receive larger similarity values, while distant points receive smaller ones. This is the basic intuition behind the probability-based neighborhood descriptions used by methods such as t-SNE, and also behind the weighted-neighbor view used by UMAP.

5.2 Why "Local Structure" Matters

When a dataset has nonlinear geometry, global Euclidean coordinates can be misleading. Two points may be far apart in the ambient high-dimensional space yet still belong to the same curved surface or local pattern; conversely, a linear projection may mix together points that should remain separated.

This motivates the distinction between **local structure** and **global structure**:

- **Local structure:** who the nearest neighbors of each point are; how small neighborhoods are organized.
- **Global structure:** large-scale geometry, such as distances between faraway groups, overall orientation, and large-scale shape.

PCA pays more attention to global variance structure. By contrast, t-SNE and UMAP are mainly designed to preserve local structure. In practical terms, this means that if point A is among the important neighbors of point B in the original space, the embedding should place them near each other in 2D or 3D.

Neighborhood preservation

Neighborhood preservation means that points that are close to one another in the original space should remain close, or at least strongly associated, in the low-dimensional embedding.

Figure 7 illustrates the distinction between preserving *local structure* and preserving *global geometry*. In the original space, the data may lie on a curved structure, and each point has a meaningful neighborhood around it. In the embedding, the algorithm tries to keep those neighborhoods coherent, but it may reorganize the overall geometry substantially. This is why nonlinear embeddings are useful for revealing local grouping patterns, yet should not be read as literal maps of all pairwise distances.

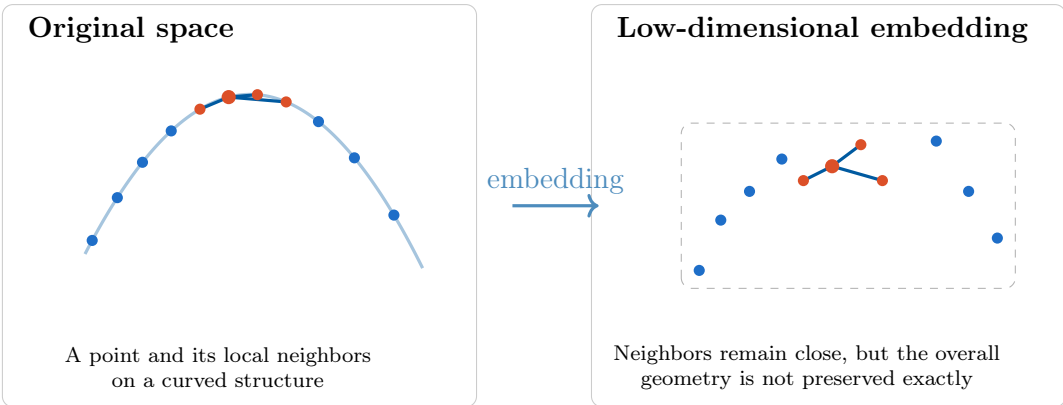


Figure 7: Local neighborhood preservation versus global geometric preservation. In nonlinear dimensionality reduction, the main goal is often to keep nearby points close in the embedding, even if the large-scale geometry is altered. This is one reason why distances between far-apart groups in a t-SNE or UMAP plot should be interpreted with caution.

This focus makes nonlinear embeddings especially useful for visualization. If the original data contain groups, subgroups, or curved clusters, preserving neighborhood structure can reveal those patterns more clearly than a linear projection.

Preserving local neighborhoods does *not* imply preserving all distances faithfully. In particular, the distance between two visible clusters in a t-SNE or UMAP plot should not be interpreted too literally unless there is additional evidence that the method preserved that global relationship well.

5.3 A Probability View of Neighborhoods

One elegant way to represent neighborhood structure is through probabilities. Instead of saying only that two points are "close" or "far", we assign a *probability-like weight* that indicates how strongly one point regards another as a neighbor.

For a fixed point i, imagine distributing a total mass of 1 among all the other points:

- points very close to i receive larger mass;

- moderately close points receive some mass;
- distant points receive almost none.

This creates a probability distribution over neighbors of point i. Such a distribution has an important advantage: it expresses *relative neighborhood importance*. The method is no longer asking only for absolute distances, but rather:

> Which other points matter most as neighbors of this point?

This is the perspective taken by t-SNE. In the high-dimensional space, each point defines a distribution over its neighbors. In the low-dimensional space, the candidate embedding defines another distribution. The algorithm then tries to make those two distributions as similar as possible.

Why probabilities can help

Probability distributions are a natural language for "soft neighborhoods." They avoid a rigid yes/no decision about whether two points are neighbors and instead express neighborhood strength gradually.

5.4 A Minimal Intuition for KL Divergence

Once neighborhood relationships are represented probabilistically, we need a way to compare the high-dimensional neighborhood distribution with the low-dimensional one. In t-SNE, this role is played by the **Kullback–Leibler (KL) divergence**.

Informal interpretation of KL divergence

The **KL divergence** measures how different one probability distribution is from another. It is zero only when the two distributions match exactly, and it grows as they disagree more.

In the context of t-SNE, the key practical interpretation is the following:

- if two points are strong neighbors in the original space, but the embedding places them far apart, the loss increases substantially;
- if two points are weak neighbors in the original space, the algorithm is less worried about their exact relative positions in the embedding.

So even without mastering the formal properties of KL divergence, students should understand its role: it tells t-SNE to prioritize the faithful preservation of important local relationships.

What t-SNE is optimizing

t-SNE builds one probability-based description of neighborhood structure in the original space and another in the low-dimensional space, then moves the low-dimensional points so that those two descriptions become as similar as possible.

5.5 Why a Low-Dimensional Map Is Hard: The Crowding Issue

A major difficulty in nonlinear embedding is that a high-dimensional neighborhood cannot be represented perfectly in 2D. In high dimensions, a point may have many different neighbors at different moderate distances. In a two-dimensional plot, there is limited "room" to place all those neighbors while preserving every relationship.

This is one reason dimensionality reduction for visualization is inherently approximate. The algorithm must decide which aspects of structure to prioritize. t-SNE explicitly prioritizes local neighborhood preservation, even at the expense of some global geometric faithfulness.

This explains why a t-SNE or UMAP plot should be interpreted as a *map of neighborhood relations*, not as a literal scaled drawing of the original geometry.

5.6 The Informal Idea of a Manifold

UMAP is motivated by the idea that high-dimensional data may lie near a much lower-dimensional curved surface. This motivates the notion of a **manifold**.

Informal notion of manifold

A **manifold** is a space that may look curved globally, but that looks approximately flat when examined in a sufficiently small neighborhood.

A familiar analogy is the surface of the Earth. Globally, it is curved. But in a small enough region, it appears almost flat. Similarly, a dataset may live in a very high-dimensional ambient space while still being organized around a lower-dimensional curved structure.

For example, images of the same object under different rotations, lighting conditions, or small deformations may occupy only a tiny structured subset of the full pixel space. Although each image may have thousands of pixel dimensions, the true degrees of freedom generating the data can be much fewer.

This manifold intuition helps explain why nonlinear dimensionality reduction can succeed where linear projection may fail. If the data lie on a curved surface, a linear method may cut across that surface in an unhelpful way, whereas a neighborhood-based nonlinear method may better respect its local organization.

Figure 8 gives an informal geometric intuition for the idea of a manifold. The full dataset may follow a curved structure when viewed globally, but each small region may still look approximately flat. This helps explain why neighborhood-based nonlinear methods can succeed even when a single global linear projection is inadequate: they rely more on the organization of local patches than on one global coordinate system.

5.7 Graphs as a Representation of Local Geometry

Another useful idea, especially for UMAP, is to represent the dataset as a graph:

- each data point becomes a node;
- edges connect points that are considered neighbors;
- edge weights indicate how strong the neighborhood relation is.

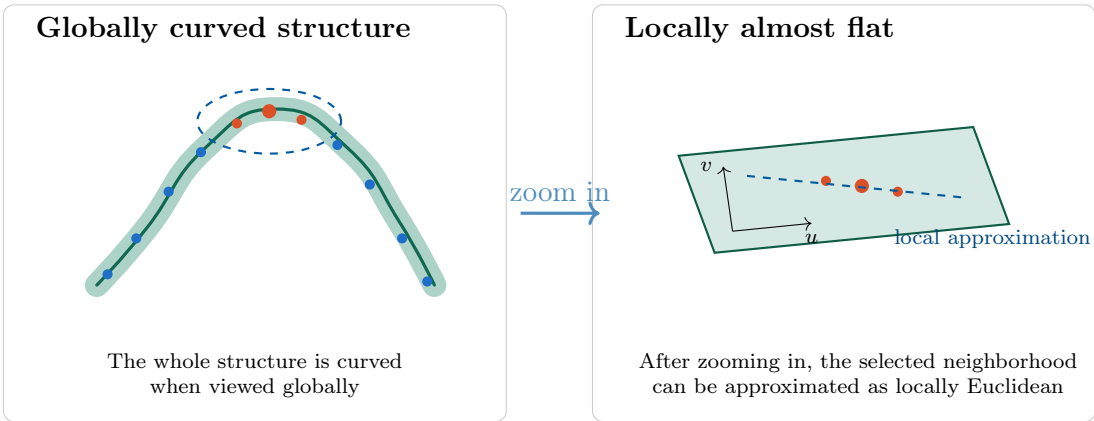


Figure 8: Informal manifold intuition. A dataset may lie on a globally curved structure embedded in a high-dimensional space, while each sufficiently small neighborhood can be approximated by a nearly flat local patch. This local-flatness intuition helps motivate neighborhood-based nonlinear methods such as UMAP.

This graph-based view is powerful because it captures local geometry without requiring a linear coordinate system. Once such a graph is built in the original space, the algorithm searches for a low-dimensional arrangement whose own graph structure resembles it as closely as possible.

This is the basic intuition behind UMAP. The technical details involve fuzzy weighted graphs and topological arguments, but the central idea is simpler: *construct a map whose local connectivity pattern resembles the original one.*

5.8 What This Background Changes in Practice

This background leads to three practical reading rules for the rest of these notes:

1. For **PCA**, think in terms of variance, orthogonal directions, and projection.
2. For **t-SNE**, think in terms of probability-based neighborhood preservation.
3. For **UMAP**, think in terms of local graph structure and manifold organization.

Keeping these three viewpoints separate helps avoid a common confusion: students sometimes assume that all dimensionality-reduction methods are trying to preserve the same notion of structure. They are not. PCA emphasizes linear variance structure; t-SNE and UMAP emphasize local neighborhood structure in a nonlinear embedding.

Entry Ticket 8

Why is it reasonable for a nonlinear visualization method to preserve neighborhoods rather than trying to preserve all pairwise distances exactly? What kinds of information are likely to be lost when we compress a high-dimensional dataset into only two dimensions?

6 t-SNE: t-Distributed Stochastic Neighbor Embedding

6.1 Motivation: The Limits of Linear Methods

So far, we have studied PCA, which is *linear*: it can only find directions that are straight lines through the original space. Many real datasets, however, have nonlinear structure. Consider a dataset of handwritten digits: the meaningful variation (pen angle, loop size, etc.) does not live on a linear subspace but on a curved, low-dimensional surface (*manifold*) embedded in the high-dimensional pixel space.

PCA will project all digits onto straight lines through pixel space, potentially mixing together images that are visually similar but lie in different parts of the manifold. What we would ideally want is a method that "unfolds" the manifold and represents it in 2D or 3D in a way that preserves local neighborhood relationships.

t-SNE (t-Distributed Stochastic Neighbor Embedding), introduced by van der Maaten and Hinton [2008], is a nonlinear dimensionality reduction method designed specifically for this purpose. It is one of the most widely used tools for *visualization* of high-dimensional data.

6.2 The Core Idea

t-SNE works in two steps.

Step 1: Modeling similarity in the high-dimensional space. For each pair of points i and j in the original n-dimensional space, t-SNE computes a similarity p_{ij}, which measures how likely it is that i and j would be considered "neighbors". This similarity is defined using a Gaussian (normal) kernel centered at each point:

$$p_{j|i} = \frac{\exp\bigl(-\|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2/(2\sigma_i^2)\bigr)}{\sum_{k\neq i}\exp(-\|\mathbf{x}^{(i)} - \mathbf{x}^{(k)}\|^2/(2\sigma_i^2))}, \qquad p_{ij} = \frac{p_{j|i} + p_{i|j}}{2m}.$$

Intuitively, p_{ij} is large when $\mathbf{x}^{(i)}$ and $\mathbf{x}^{(j)}$ are close together, and small when they are far apart. The bandwidth σ_i is chosen adaptively for each point (see Section 6.4).

Step 2: Modeling similarity in the low-dimensional space. For the corresponding low-dimensional representations $\mathbf{y}^{(1)}, \ldots, \mathbf{y}^{(m)} \in \mathbb{R}^2$ (which are what we want to find), t-SNE computes analogous similarities q_{ij}. Crucially, instead of a Gaussian kernel, t-SNE uses a **Student's t-distribution with one degree of freedom** (a Cauchy distribution):

$$q_{ij} = \frac{\bigl(1 + \|\mathbf{y}^{(i)} - \mathbf{y}^{(j)}\|^2\bigr)^{-1}}{\sum_{k\neq\ell}\bigl(1 + \|\mathbf{y}^{(k)} - \mathbf{y}^{(\ell)}\|^2\bigr)^{-1}}.$$

Step 3: Matching the two distributions. t-SNE finds the low-dimensional positions $\{\mathbf{y}^{(i)}\}$ by minimizing the **Kullback–Leibler (KL) divergence** between the high-dimensional similarity distribution $P = \{p_{ij}\}$ and the low-dimensional similarity distribution $Q = \{q_{ij}\}$:

$$\mathcal{L} = \mathrm{KL}(P\|Q) = \sum_{i\neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}.$$

This is minimized via gradient descent. Intuitively, the algorithm tries to arrange the 2D points $\mathbf{y}^{(i)}$ so that nearby pairs in the original space are also nearby in 2D, and distant pairs in the original space are also distant in 2D.

6.3 Why the t-Distribution?

The use of the t-distribution in the low-dimensional space is not arbitrary. It is the key technical insight of t-SNE, introduced to address the **crowding problem**.

In high dimensions, a typical point has many neighbors at moderate distances. When we try to represent this point in 2D, there is simply not enough room to place all moderately-distant neighbors at moderate distances in 2D. A Gaussian kernel in the low-dimensional space would force many points to cluster tightly together, producing distorted visualizations.

The t-distribution has *heavier tails* than a Gaussian. This means that two points can be far apart in 2D and still have a non-negligible similarity q_{ij}. As a consequence, t-SNE can "spread out" groups of points in 2D without being penalized by the loss function, alleviating crowding.

Gaussian vs. t-distribution tails

For a standard Gaussian, the probability of a value greater than 3 standard deviations is about 0.13%. For a t-distribution with one degree of freedom (Cauchy), the corresponding probability is about 10%. This difference in tail behavior is what gives t-SNE its ability to represent high-dimensional neighborhoods faithfully in 2D.

6.4 Perplexity: Controlling the Notion of Neighborhood

The bandwidth σ_i in the high-dimensional similarity is not set manually. Instead, it is chosen so that the conditional distribution $p_{\cdot|i}$ has a fixed **perplexity**:

$$\text{Perplexity}(P_i) = 2^{H(P_i)},$$

where $H(P_i) = -\sum_j p_{j|i} \log_2 p_{j|i}$ is the Shannon entropy of the conditional distribution. The perplexity can be interpreted roughly as the effective number of neighbors each point considers. Typical values are between 5 and 50.

- A **low perplexity** (e.g., 5) makes each point consider only a very small local neighborhood. The algorithm focuses on fine-grained local structure and may miss broader patterns.
- A **high perplexity** (e.g., 50) makes each point consider a larger neighborhood. The algorithm captures more global structure, but local clusters may appear less well-separated.

Perplexity is the most important hyperparameter of t-SNE and should be tuned for each dataset. It is often a good idea to run t-SNE with several different perplexity values and compare the resulting visualizations.

6.5 Properties and Limitations

What t-SNE does well:

- Produces visually compelling 2D or 3D visualizations that reveal cluster structure in high-dimensional data.

- Preserves local neighborhood relationships much better than PCA.
- Handles curved, manifold-like data structures.

Important limitations:

- **Non-parametric:** Standard t-SNE is usually used as a non-parametric visualization method and does not naturally provide a reusable transform for new data. To embed a new point, the entire optimization must be rerun.
- **Slow:** the computation of pairwise similarities costs $O(m^2)$ time and memory. For large datasets ($m > 10\,000$), approximations (e.g., the Barnes–Hut method) are needed.
- **Non-deterministic:** the gradient descent optimization starts from a random initialization. Different runs with the same hyperparameters may produce different layouts, though the cluster structure is usually preserved.
- **Global structure not preserved:** t-SNE is optimized to preserve local neighborhoods. The distances *between* clusters in a t-SNE plot are not necessarily meaningful. Do not interpret the size or distance between clusters as representing true distances in the original space.
- **Not suitable as a preprocessing step for other models:** because t-SNE is non-parametric and produces a non-repeatable transformation, it is almost exclusively used for visualization, not as a feature-extraction step for a downstream classifier.

How to interpret t-SNE plots

t-SNE plots are most useful for exploring whether local neighborhoods form visually coherent groups. If points that were neighbors in the original high-dimensional space appear close together in the 2D embedding, the plot is showing a meaningful local pattern.
However, a t-SNE plot should not be interpreted as a faithful geometric map of the original data. In particular:

- distances between faraway clusters should not be interpreted literally;
- the relative sizes of clusters are not reliable indicators of how much variation or diversity they contain;
- apparent densities in the plot may be artifacts of the embedding procedure;
- the axes of the plot do not have a direct semantic interpretation.

Thus, t-SNE is best viewed as an exploratory visualization tool. It can suggest interesting neighborhood patterns, but those patterns should be checked with additional analyses before being treated as substantive conclusions.

Entry Ticket 9

A t-SNE plot shows three clearly separated clusters. What conclusions can you reasonably draw from this visualization? What conclusions would require additional evidence?

7 UMAP: Uniform Manifold Approximation and Projection

7.1 Motivation and Overview

UMAP (Uniform Manifold Approximation and Projection) [McInnes et al., 2018] is a more recent nonlinear dimensionality reduction method that addresses several of the practical limitations of t-SNE. Like t-SNE, UMAP is designed to handle nonlinear manifold structure. Compared with standard t-SNE, UMAP is often faster, can preserve more large-scale structure in many applications, and provides a way to transform new data after the initial fit.

UMAP and global structure

UMAP embeddings often provide a better visual impression of large-scale organization than t-SNE embeddings, especially when appropriate values of `n_neighbors` are used. However, distances between faraway clusters in a UMAP plot should still be interpreted with care. UMAP is primarily a neighborhood-preserving method, not a method for faithfully preserving all global distances.

The theoretical foundation of UMAP draws on **Riemannian geometry** and **algebraic topology**, specifically on the theory of simplicial complexes and *fuzzy topological representations*. While these foundations are mathematically sophisticated, the practical behavior of UMAP can be understood through a high-level intuition analogous to t-SNE.

7.2 Core Intuition

Like t-SNE, UMAP seeks to represent high-dimensional data in a low-dimensional space while preserving neighborhood structure. Its approach can be understood in three phases:

Phase 1: Building a fuzzy graph in the high-dimensional space. UMAP constructs a weighted graph where the nodes are the data points. Two points are connected by an edge whose weight reflects how likely they are to be mutual neighbors. These weights are computed using a notion of "fuzzy" neighborhood: close pairs get weights near 1, distant pairs get weights near 0.

Manifold intuition behind UMAP

At a high level, UMAP is motivated by the idea that high-dimensional data may lie near a lower-dimensional structure embedded in the original feature space. This lower-dimensional structure is often informally called a *manifold*.
The practical role of this idea is the following: instead of trying to preserve all pairwise distances, UMAP builds a graph of local neighborhoods. Each point is connected more strongly to nearby points, and these local connections are then used to construct a low-dimensional embedding.
The technical theory behind UMAP uses ideas from Riemannian geometry and topological data analysis. For these notes, however, the most important point is simpler: UMAP tries to preserve the organization of local neighborhoods because those neighborhoods are treated as evidence of the underlying geometric structure

of the data.

Technical nuance

Some descriptions of UMAP state that the data are assumed to be uniformly distributed on a Riemannian manifold. This should not be read as a simple empirical requirement that the observed dataset must look uniform. Rather, it is part of the mathematical construction used to define local distances and neighborhood strengths. In practice, UMAP adapts neighborhood scales locally, so dense and sparse regions may both contribute to the embedding.

Phase 2: Building a fuzzy graph in the low-dimensional space. UMAP constructs an analogous weighted graph for the low-dimensional layout. The weights in this graph depend on the Euclidean distances between the low-dimensional points.

Phase 3: Matching the two graphs. UMAP now has two weighted neighborhood structures: one built from the original high-dimensional data and another induced by the current positions of the points in the low-dimensional embedding. The goal of the optimization step is to adjust the low-dimensional positions so that these two fuzzy graphs become as similar as possible.

This objective may look familiar from our previous discussions of logistic regression and model calibration. In those contexts, cross-entropy measured the mismatch between true labels and predicted probabilities. Here, the same general idea appears in a geometric form: cross-entropy measures the mismatch between two weighted neighborhood structures.

Let w_{ij}^{high} denote the strength of the connection between points i and j in the high-dimensional graph, and let w_{ij}^{low} denote the corresponding connection strength induced by their positions in the low-dimensional embedding. UMAP optimizes an objective related to the cross-entropy between these two fuzzy graphs:

$$\mathcal{L} = \sum_{i,j} \left[w_{ij}^{\text{high}} \log \frac{w_{ij}^{\text{high}}}{w_{ij}^{\text{low}}} + \left(1 - w_{ij}^{\text{high}}\right) \log \frac{1 - w_{ij}^{\text{high}}}{1 - w_{ij}^{\text{low}}} \right].$$

The interpretation is analogous to the classification case: the loss becomes large when the representation being learned disagrees with the reference structure. Here, however, the "reference" is not a class label. It is the high-dimensional neighborhood graph. The "prediction" is not a class probability. It is the neighborhood strength induced by the current low-dimensional positions.

Thus, UMAP uses cross-entropy to answer a geometric question: do the neighbor relationships in the embedding match the neighbor relationships in the original space? If two points are strongly connected in the high-dimensional graph, the optimization encourages them to be close in the embedding. If two points are weakly connected in the high-dimensional graph, the optimization discourages the embedding from placing them too close together.

In practice, this optimization is performed approximately using stochastic gradient descent. Like t-SNE, UMAP uses a heavy-tailed similarity function in the low-dimensional

space, which helps alleviate the crowding problem by making it easier to separate points that should not be close neighbors.

7.3 Key Hyperparameters

UMAP has two main hyperparameters:

- `n_neighbors`: the number of nearest neighbors used to construct the high-dimensional graph. This plays a role analogous to perplexity in t-SNE. Small values focus on very local structure; large values allow the algorithm to capture more global patterns. Typical values range from 5 to 50. Increasing `n_neighbors` generally produces a more connected, globally coherent layout.
- `min_dist`: the minimum distance allowed between points in the low-dimensional layout. Small values (e.g., 0.0) allow UMAP to cluster points very tightly, producing compact, well-separated clusters. Larger values (e.g., 0.5) spread points out more, preserving more of the global topology at the cost of less tight clusters.

As with t-SNE, it is good practice to explore several combinations of these hyperparameters before drawing conclusions from a UMAP visualization.

7.4 Comparison with PCA and t-SNE

Table 1 summarizes the main similarities and differences among the three dimensionality-reduction methods covered in these notes. In particular, it highlights that the methods differ not only in whether they are linear or nonlinear, but also in the *type of output* they produce: PCA yields a linear projection onto principal components, whereas t-SNE and UMAP produce nonlinear low-dimensional embeddings.

As Table 1 shows, PCA, t-SNE, and UMAP should not be viewed as interchangeable tools. PCA produces a *linear projection* onto orthogonal directions of maximum variance, which makes it especially attractive for preprocessing, compression, and interpretation of variance structure. By contrast, t-SNE and UMAP produce *nonlinear embeddings*, whose main goal is to preserve neighborhood relationships in a low-dimensional space.

A few remarks on this comparison:

- UMAP is generally faster than t-SNE by a significant margin, especially for large datasets. This makes it practical as an exploratory tool even for datasets with hundreds of thousands of points.
- UMAP often preserves more large-scale organization than standard t-SNE, especially when `n_neighbors` is chosen appropriately. Even so, distances between far-away clusters in a UMAP plot should still be interpreted with caution. As indicated in Table 1, the relative positions of different clusters tend to carry more information in a UMAP plot than in a t-SNE plot, though they should still be interpreted with caution.
- Many UMAP implementations provide a `transform` operation that maps new observations into an existing embedding..
- Both t-SNE and UMAP produce non-deterministic results by default, since they rely on random initialization. To obtain reproducible results, set the `random_state`

Property	**PCA**	**t-SNE**	**UMAP**
Linear/Nonlinear	Linear	Nonlinear	Nonlinear
Type of output	Linear projection	Nonlinear embedding	Nonlinear embedding
Main preservation goal	Global variance	Local neighborhoods	Local neighborhoods
Large-scale structure	Linear variance structure	Usually unreliable	Often more informative, but still approximate
Transforms new data naturally?	Yes	Usually no	Yes, in many implementations
Typical use case	Preprocessing, compression, visualization	Exploratory visualization	Visualization, sometimes preprocessing
Deterministic by default?	Yes	No	No

Table 1: Comparison of PCA, t-SNE, and UMAP. PCA provides a linear projection based on variance, whereas t-SNE and UMAP produce nonlinear embeddings focused mainly on neighborhood preservation.

(or `random_seed`) parameter.

7.5 Implementation in Python

UMAP is available through the `umap-learn` package (not included in Scikit-Learn by default). Its API is similar to Scikit-Learn transformers:

- `n_neighbors`: controls locality vs. globality trade-off (default: 15).
- `n_components`: number of output dimensions (default: 2).
- `min_dist`: minimum distance in the low-dimensional space (default: 0.1).
- `metric`: distance metric for the high-dimensional space (default: `'euclidean'`).
- `random_state`: for reproducibility.

When UMAP is used as a transformer in a predictive workflow, it should be fitted on training data only and then applied to validation/test data using the fitted transform.

When to use UMAP vs. t-SNE vs. PCA

As a practical guideline: (1) use **PCA** as a default first step for high-dimensional numeric data, especially when preprocessing for a downstream model or when ex-

plained variance is meaningful; (2) use **t-SNE** or **UMAP** when you want to explore local neighborhood structure visually in 2D or 3D and the data may have nonlinear structure; (3) consider **UMAP** instead of standard t-SNE when the dataset is large, when you want a somewhat more informative large-scale layout, or when you need a reusable `transform` operation.

Entry Ticket 10

UMAP and t-SNE are both described as methods that preserve "local neighborhood structure." What does this mean in practice? Can you think of a type of question about the data that could be answered from a UMAP or t-SNE visualization but *not* from a PCA plot? Conversely, what information does a PCA plot convey that a t-SNE or UMAP plot does not?

8 Dimensionality Reduction in a ML Pipeline

Dimensionality reduction is often not used in isolation. Instead, it may be inserted into a pipeline before a downstream learner such as logistic regression, k-NN, or a support vector machine. This is especially common with PCA, and it can also be done with UMAP when it is being used as a reusable transformer. By contrast, t-SNE is used primarily for visualization and exploratory analysis, not as a standard preprocessing step for predictive models.

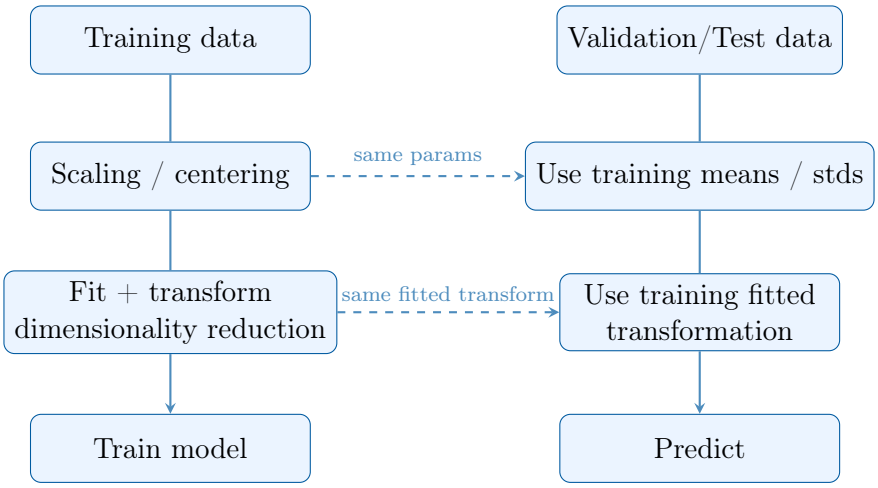


Figure 9: Correct pipeline usage for dimensionality reduction methods that define a reusable transformation, such as PCA and UMAP. Scaling parameters and the dimensionality-reduction mapping must be learned on the training set only, then applied unchanged to validation or test data. Dashed arrows indicate parameter reuse.

Figure 9 highlights an important data leakage issue. If the validation or test set were used to fit the dimensionality-reduction step, information from those data would leak into training, producing overly optimistic evaluation results.

Any dimensionality-reduction method that learns its transformation from the data must be *fitted* on the training data only, and then *applied* unchanged to validation and test sets. In cross-validation, the method must be refitted inside each training fold. This is exactly the same principle that applies to feature scaling and encoding. For **PCA**, this fitted object is the projection matrix. For **UMAP**, it is the learned embedding transformation used by `transform`. Although the same leakage principle conceptually applies to **t-SNE**, t-SNE is usually not used in this way, because it is primarily a visualization method rather than a standard reusable preprocessing transform for downstream prediction.

9 Summary

The methods discussed in these notes all reduce dimensionality, but they do so with different goals, assumptions, and practical uses. PCA is a linear method with a clear variance-maximization interpretation. t-SNE and UMAP are nonlinear embedding methods focused mainly on preserving neighborhood relationships, especially for visualization.

It is therefore misleading to think of PCA, t-SNE, and UMAP as interchangeable tools. They all produce lower-dimensional representations, but they optimize different criteria and should be interpreted differently.

Method	**Main goal**	**What it tries to preserve**	**Typical use**
PCA	Find a lower-dimensional linear representation.	Global variance and projection geometry.	Preprocessing, compression, denoising, and sometimes visualization.
t-SNE	Build a nonlinear low-dimensional visualization.	Local neighborhood relationships.	Exploratory 2D or 3D visualization of possible groups.
UMAP	Build a nonlinear neighborhood-preserving embedding.	Local neighborhoods, with some large-scale structure often retained.	Visualization and, in some workflows, nonlinear preprocessing.

Table 2: Conceptual comparison of PCA, t-SNE, and UMAP. PCA is a linear variance-preserving method, whereas t-SNE and UMAP are nonlinear neighborhood-preserving methods.

Key Takeaways

1. Dimensionality reduction is useful for reducing redundancy, noise, computational cost, and the difficulty of working with high-dimensional data.

Question	**PCA**	**t-SNE**	**UMAP**
Is the method linear?	Yes	No	No
Can it naturally transform new data?	Yes	Usually no	Yes, in many implementations
Are the axes directly interpretable?	Sometimes	No	No
Can it be used as preprocessing for a predictive model?	Yes	Usually no	Sometimes
Is it deterministic by default?	Yes	No	No
Is it sensitive to hyperparameters?	Moderate	High	High
Should distances between faraway clusters be trusted?	Only when a linear projection is appropriate	No	With care

Table 3: Practical comparison of PCA, t-SNE, and UMAP. These differences matter when dimensionality reduction is used for visualization, preprocessing, compression, or exploratory analysis.

2. PCA is a **linear feature-extraction** method. It finds orthogonal directions of maximum variance and projects the data onto the first few of these directions.
3. Centering is essential for PCA. Standardization is also important when features are measured on different numerical scales.
4. Explained variance is useful for understanding how much structure PCA retains, but it is not the same as predictive relevance. A low-variance direction may still be important for predicting the target variable.
5. When PCA or UMAP is used inside a predictive pipeline, it must be fit only on the training data. Validation and test data should only be transformed using the fitted preprocessing steps.
6. t-SNE and UMAP are **nonlinear embedding** methods. They are designed mainly to preserve local neighborhood relationships, not to produce interpretable axes.
7. t-SNE is best understood as an exploratory visualization tool. Cluster separation may reveal interesting local structure, but distances between faraway clusters, cluster sizes, and densities should not be interpreted literally.
8. UMAP shares the neighborhood-preserving spirit of t-SNE, but is often faster than standard t-SNE and many implementations provide a way to transform new data into an existing embedding.
9. A practical rule is: use PCA for simple, linear, reusable compression; use t-SNE for exploratory visualization of local neighborhoods; and use UMAP

when you want a nonlinear neighborhood-preserving embedding that may also be reusable in a pipeline.

9 References

Harold Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441, 1933. doi: 10.1037/h0071325.

Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.

Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008.